



LABORATORIO 3: BASES DE DATOS NOSQL CON MONGODB



**BASES DE
DATOS
AVANZADAS
2026-1**

GESTOR DE CLANES Y RAIDS PARA MMORPG

INTEGRANTES:

- **SEBASTIÁN SALLES**
- **MARTÍN FUENTES**
- **DIEGO VEGA**

PROFESOR: MATÍAS CALDERÓN
AYUDANTE: FERNANDO SOLÍS



CONTENIDO

01

PORTADA

02

CONTENIDO

03

INTRODUCCIÓN

04

AVANCE

05

ARQUITECTURA

06

FUNCIONALIDADES

07

MODELADO DE DATOS

08

VALIDACIÓN DE ESQUEMA

09

TRANSACCIÓN MULTI-DOCUMENTO

10

AGGREGATION PIPELINE

11

INDICES

12

CHANGE STREAMS

13

CONTRIBUCIONES

14

CONCLUSIONES

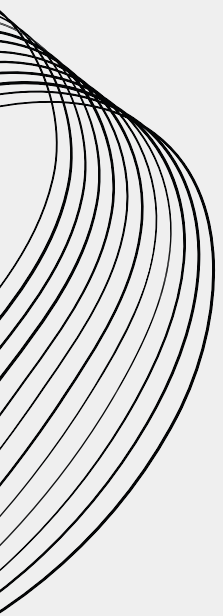
INTRODUCCIÓN



Entrega de tercera iteración, cuyo objetivo es desarrollar una para gestionar jugadores, sus personajes, eventos (Raids), asistencia y distribución de botín (Loot).

Basado en entregas anteriores, en esta iteración se hizo una migración desde una base de datos relacional SQL a una NOSQL, específicamente MongoDB.

Se trabaja con recursos como Replica Set, Schema Validation, indexación y Aggregation Pipelines. Todo esto en un contexto de desarrollo completo frontend → backend → base de datos.



AVANCE DEL PROYECTO

Laboratorio 1

PostgreSQL:
Modelo relacional,
procedimientos y
triggers

Laboratorio 2

PostgreSQL + PostGIS:
Capa geoespacial (usos
de coordenadas
geoespaciales)

Laboratorio 3

MongoDB:
Modelo
documental,
transacciones,
agregación,
reactividad

Cambio del motor de base de datos, pasamos de triggers y procedimientos a validadores, transacciones y change streams.



ARQUITECTURA

1. Capa de Presentación

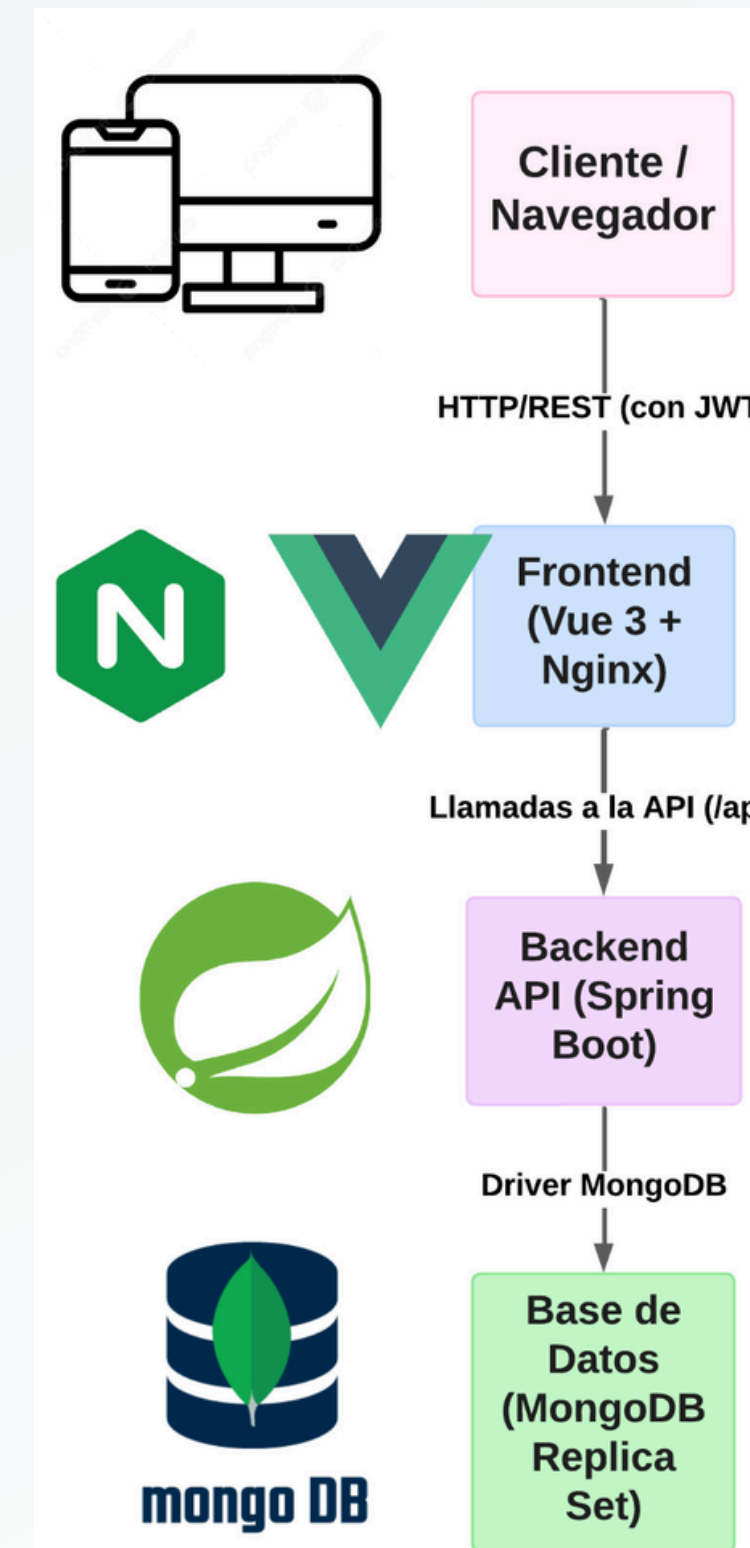
Interfaz de usuario reactiva desarrollada en Vue 3 y servida a través de Nginx. Se comunica con el servidor mediante peticiones seguras (JWT).

2. Lógica de Negocio (API)

Backend construido con Spring Boot. Expone endpoints REST, valida reglas de negocio y orquesta la comunicación segura hacia la persistencia.

3. Persistencia de Datos

Clúster de MongoDB configurado en Replica Set. Esta topología garantiza alta disponibilidad, redundancia de datos y recuperación automática ante fallos de nodos primarios.



FUNCIONALIDADES

A continuación se mostrarán cada una de las funcionalidades/requisitos solicitados en el enunciado del laboratorio

- 1. MODELADO DE DATOS**
- 2. VALIDACIÓN DE ESQUEMA**
- 3. TRANSACCIÓN MULTI-DOCUMENTO**
- 4. AGGREGATION PIPELINE**
- 5. ÍNDICES**
- 6. CHANGE STREAMS**

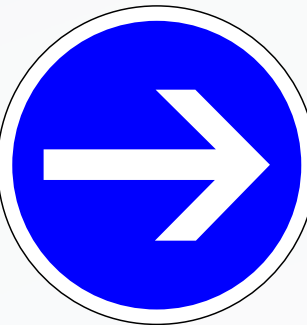
1. MODELADO DE DATOS

"Justifique e implemente si los personajes deben embeberse dentro del documento del jugador o referenciarse en una colección propia, considerando que cada personaje participa en múltiples raids de forma independiente."

Cada personaje participa en muchas raids de forma independiente.

Personajes embebidos en el jugador habría que duplicarlo por cada raid.

Por lo que se tomó la decisión de referenciar.



```
// 3. Personajes
db.createCollection("personajes", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["_id", "idUsuario", "nombrePersonaje", "clase", "faccion", "rolClan", "nivel", "itemLevel", "puntosDkpActuales"],
      properties: {
        _id: { bsonType: "int" },
        idUsuario: { bsonType: "int" },
        idClan: { bsonType: ["int", "null"] },
        nombrePersonaje: { bsonType: "string" },
        clase: { bsonType: "string" },
        faccion: { bsonType: "string" },
        rolClan: { bsonType: "string" },
        nivel: { bsonType: "int", minimum: 1, maximum: 60 },
        itemLevel: { bsonType: "int", minimum: 0, maximum: 1000 },
        puntosDkpActuales: { bsonType: "int", minimum: 0 },
        caido: { bsonType: "bool" },
        inventario: {
          bsonType: "object",
          required: ["idInventario"],
          properties: {
            idInventario: { bsonType: "int" },
            armaduraEquipado: { bsonType: ["int", "null"] },
            armaEquipado: { bsonType: ["int", "null"] },
            accesorioEquipado: { bsonType: ["int", "null"] },
            items: { bsonType: "array", items: { bsonType: "int" } }
          }
        }
      }
    }
  }
})
```


2.VALIDACIÓN DE ESQUEMA

"Diseñe una regla de Schema Validation que impida asignar un ítem de Loot a un personaje que no participó en la Raid, o que ya se encuentre "caído" (fuera de combate) al momento de la distribución."

```
// 6. Loot Pool
db.createCollection("loot_pool", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["_id", "idPersonaje", "idItem", "idRaid", "participoRaid", "personajeCaído", "canjeado", "fecha"],
      properties: {
        _id: { bsonType: "int" },
        idPersonaje: { bsonType: "int" },
        idItem: { bsonType: "int" },
        idRaid: { bsonType: "int" },
        participoRaid: { bsonType: "bool", description: "El personaje debe haber participado en la raid", enum: [true] },
        personajeCaído: { bsonType: "bool", description: "El personaje no debe estar caído", enum: [false] },
        canjeado: { bsonType: "bool" },
        fecha: { bsonType: "date" }
      }
    }
  }
});
```

3. TRANSACCIÓN MULTI-DOCUMENTO

"En un evento de Raid, ¿cómo diseñaría una transacción que distribuya el Loot entre varios personajes de forma atómica, evitando que un mismo ítem quede asignado a más de un personaje ante solicitudes concurrentes?"

```
// Filtramos ANTES de insertar: solo confirmados y no-caidos reciben botin.  
// El $jsonSchema de MongoDB actua como segunda barrera de seguridad.  
public void distribuirBotin(Integer idRaid) { 1usage
```

```
    // Ejecutar dentro de una transaccion con sesion explicita  
    org.springframework.transaction.support.TransactionTemplate txTemplate  
        = new org.springframework.transaction.support.TransactionTemplate(transactionManager);  
  
    txTemplate.execute( TransactionStatus status -> {  
        Random rand = new Random();  
        Set<String> itemsAsignados = new HashSet<>(); // Para evitar doble asignacion de mismo item en misma raid  
  
        for (Raid.InscripcionRaid ins : raid.getInscripciones()) {  
            // Filtrar: solo confirmados  
  
            // Insert atomico - el $jsonSchema valida como segunda barrera  
            mongoTemplate.insert(lp);  
        }  
    });  
}
```

4. AGGREGATION PIPELINE

"Construya un pipeline que calcule el ranking de clanes según el desempeño en raids (tiempo de finalización, asistencia, daño total) usando \$group, \$lookup y \$sort."

```
public void refrescarRankingClanes() { 2 usages
    java.util.List<org.bson.Document> pipeline = java.util.Arrays.asList(
        new org.bson.Document("$match", new org.bson.Document("estado", "COMPLETADA")),
        new org.bson.Document("$unwind", "$inscripciones"),
        new org.bson.Document("$match", new org.bson.Document("inscripciones.confirmado", true)),
        new org.bson.Document("$group", new org.bson.Document()
            .append("_id", "$idClan")
            .append("asistencia_total", new org.bson.Document("$sum", 1))
            .append("dano_total", new org.bson.Document("$sum",
                new org.bson.Document("$ifNull", java.util.Arrays.asList("$inscripciones.dano", 0))))
            .append("tiempo_promedio", new org.bson.Document("$avg",
                new org.bson.Document("$ifNull", java.util.Arrays.asList("$duracionMinutos", 0))))
            .append("raids_completadas", new org.bson.Document("$addToSet", "$_id"))),
        new org.bson.Document("$lookup", new org.bson.Document()
            .append("from", "clanes")
            .append("localField", "_id")
            .append("foreignField", "_id")
            .append("as", "clan_info"))
```

4. AGGREGATION PIPELINE

```
new org.bson.Document("$multiply", new org.bson.Document()
    .append("puntaje", -1).append("dano_total", -1).append("tiempo_promedio", 1)),
new org.bson.Document("$merge", new org.bson.Document()
    .append("into", "clanes_rankeados")
    .append("whenMatched", "replace")
    .append("whenNotMatched", "insert"))
);
mongoTemplate.getCollection(collectionName: "clanes_rankeados").deleteMany(new org.bson.Document());
mongoTemplate.getCollection(collectionName: "raids").aggregate(pipeline).toCollection();
}
```

5. ÍNDICES

"Diseñe índices compuestos para filtrar rápidamente personajes disponibles por clase y rol (ej. "Healer") dentro de un clan, y un índice único sobre el nombre del personaje."

mongo-init/init.js

```
print("Creando indices...");
db.personajes.createIndex( name: { idClan: 1, clase: 1, rolClan: 1 });
db.personajes.createIndex( name: { nombrePersonaje: 1 }, keyPath: { unique: true });
db.usuarios.createIndex( name: { nombreUsuario: 1 }, keyPath: { unique: true });
db.clanes.createIndex( name: { nombreClan: 1 }, keyPath: { unique: true });
db.notificaciones.createIndex( name: { fecha: 1 }, keyPath: { expireAfterSeconds: 2592000 });
db.loot_pool.createIndex( name: { fecha: 1 }, keyPath: { expireAfterSeconds: 2592000 });
db.raids.createIndex( name: { idClan: 1, estado: 1 });
db.loot_pool.createIndex( name: { idPersonaje: 1, canjeado: 1 });
db.loot_pool.createIndex( name: { idRaid: 1, idItem: 1 }, keyPath: { unique: true });

// Índice de TEXTO para el buscador del catalogo de items (requisito de indices)
db.items.createIndex( name: { nombreItem: "text" });

// Historial por personaje, ordenado por fecha
db.historial_botin.createIndex( name: { idPersonaje: 1, fechaEntrega: -1 });
print("Indices creados.");
```


6. CHANGE STREAMS

"Implemente un listener que, al detectarse la "muerte del Boss" en la colección de eventos de Raid, dispare automáticamente el proceso de distribución de Loot y actualice una colección materializada de "clanes mejor rankeados"."

```
System.out.println("Iniciando Listener de Change Streams en coleccion 'raids'...");
mongoTemplate.getCollection( collectionName: "raids") MongoClient<Document>
    .watch() ChangeStreamIterable<Document>
    .forEach((ChangeStreamDocument<Document> change) -> {
        if (change.getOperationType() == OperationType.UPDATE) {
            org.bson.BsonDocument updatedFields = change.getUpdateDescription().getUpdatedFields();
            if (updatedFields != null && updatedFields.containsKey("estado") && "BOSS_MUERTO".equals(updatedFields.getString( key: "estado").getValue())) {
                // El documentKey contiene el _id del documento modificado
                Integer idRaid = change.getDocumentKey().getInt32( key: "_id").getValue();
                if (idRaid != null) {
                    System.out.println("Change Stream: Muerte del Boss detectada para la Raid " + idRaid);
                    try {
                        // 1. Distribuir loot (ACID transaccional, con validaciones)
                        itemRepository.distribuirBotin(idRaid);

                        // 2. Finalizar la Raid cambiando su estado a COMPLETADA
                        Raid raid = mongoTemplate.findById(idRaid, Raid.class);
                        if (raid != null) {
                            raid.setEstado("COMPLETADA");
                            mongoTemplate.save(raid);
                        }

                        // 3. Actualizar la coleccion materializada de ranking de clanes
                        itemRepository.refrescarRanking();
                        itemRepository.refrescarRankingClanes();
                        System.out.println("Change Stream: Botin distribuido y ranking de clanes refrescado.");
                    } catch (Exception ex) {
                        System.err.println("Error procesando Change Stream para Raid " + idRaid + ": " + ex.getMessage());
                    }
                }
            }
        }
    });
```

CONTRIBUCIONES

Sebastián Salles:

- Diseño e implementación de la arquitectura backend.
- Selección y configuración de bases de datos NoSQL.
- Desarrollo del código base y lógica general de la aplicación.
- Dockerización completa del entorno y despliegue.

Martín Fuentes:

- Desarrollo e implementación de la interfaz frontend.
- Integración del frontend con los servicios backend (apoyo técnico).
- Diseño de pruebas de flujo de usuario.

Diego Vega:

- Consultoría técnica y aclaración de conceptos críticos para el equipo.
- Apoyo general en tareas de backend.
- Ejecución de pruebas de flujo de usuario (testing).
- Coordinación y estructuración de la presentación final.

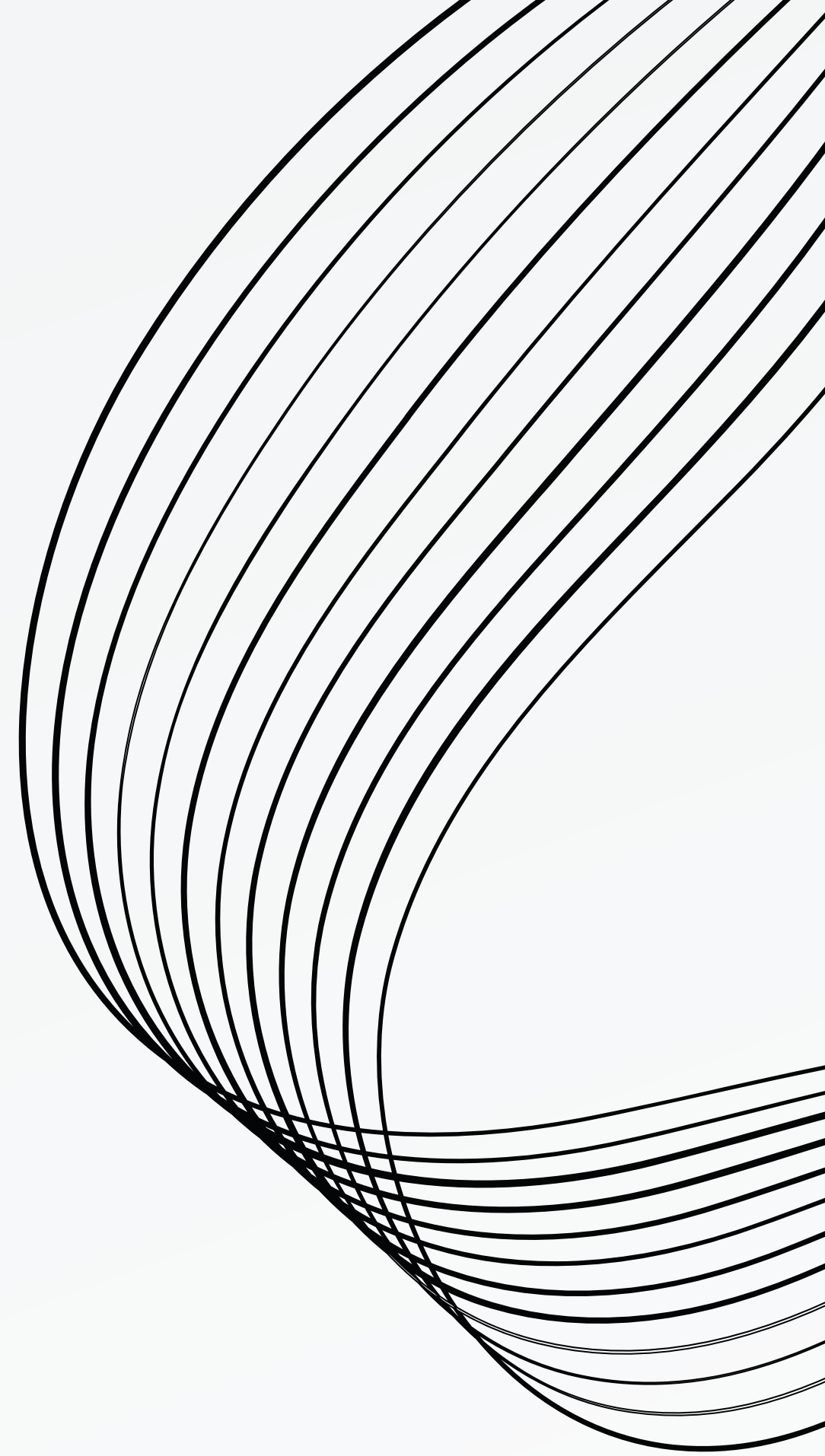
Ignacio Caro:

- Proceso de poblamiento inicial y gestión de datos de prueba.
- Adaptación y transformación de esquemas de datos SQL a formato JSON.
- Colaboración en el desarrollo del contenido para la presentación final.



CONCLUSIONES

- Se logró implementar un modelo documental, definiendo el uso de documentos embebidos y referencias según los requerimientos de cada colección.
- Se implementaron las restricciones y reglas de negocio, utilizando validaciones \$jsonSchema, transacciones e índices.
- Se configuró un Replica Set, permitiendo utilizar transacciones ACID y Change Streams en la implementación.
- Se implementó un mecanismo reactivo mediante Change Streams, permitiendo automatizar el reparto y la actualización del ranking a partir de los cambios registrados en la base de datos.





GRACIAS